# THYMELEAF.

## Sintaxis de Thymeleaf

Thymeleaf utiliza una serie de atributos específicos (prefijo th:) que se integran en las etiquetas HTML para manipular datos y controlar el flujo. Los principales atributos son:

- **th**

  : Reemplaza el texto de una etiqueta con un valor dinámico.

- **th**

  y **th**

  : Reemplazan el valor de atributos href y src con URL generadas dinámicamente.

- **th**

  : Itera sobre una colección, útil para listas y tablas.

- **th**

  y **th**

  : Condicionales para mostrar u ocultar contenido.

- **th**

  : Define un fragmento de código reutilizable en diferentes plantillas.

- **th**

  y **th**

  : Inserta un fragmento en una plantilla.

## 2. Expresiones en Thymeleaf

- **Expresiones de Variables (${})**: Permiten acceder a variables enviadas desde el controlador, por ejemplo, ${nombre}.
- **Expresiones de Selección (*{})**: Utilizan un contexto específico, muy útiles en formularios, por ejemplo, *{nombre} cuando se usa un objeto Model.

- **Expresiones de Mensajes (#{})**: Permiten acceder a mensajes internacionalizados (i18n) definidos en archivos messages.properties, por ejemplo, #{saludo}.
- **Expresiones Literales (|...|)**: Para cadenas de texto o URL, se usa | para delimitar el contenido, lo que permite que se incluyan espacios o comillas dentro, por ejemplo, |Hola ${usuario}|.

### 3. Fragmentos de Plantilla

Los fragmentos son secciones de código HTML que se pueden reutilizar en distintas plantillas, ayudando a mantener un diseño modular y fácil de mantener.

- **Definición**: Crea un fragmento en un archivo, por ejemplo, encabezado.html:

  html

  ```
  <div th:fragment="header">
     <h1>Bienvenido a mi aplicación</h1>
  </div>
  ```

- **Uso**: Puedes incluir este fragmento en otras plantillas usando:

  html

  ```
  <div th:insert="~{encabezado::header}"></div>
  ```

- th:replace reemplaza todo el contenido donde se llama con el fragmento, mientras que th:insert simplemente inserta el contenido dentro del elemento contenedor.

### 4. Formularios en Thymeleaf

Para gestionar formularios en aplicaciones Spring, Thymeleaf es compatible con Spring MVC, lo que permite que los datos se vinculen directamente con los modelos.

- **Configuración de Formulario**: Utiliza th:object para definir el objeto que recibirá los datos del formulario.
- **Campos de Formulario**: Usa th:field para los campos, permitiendo que se enlace automáticamente con las propiedades del objeto en el backend.

html

```
<form th:action="@{/guardar}" th:object="${persona}" method="post">
   <input type="text" th:field="*{nombre}" placeholder="Nombre">
```

```
    <input type="text" th:field="*{edad}" placeholder="Edad">
    <button type="submit">Guardar</button>
</form>
```

### 5. Internacionalización (i18n)

Thymeleaf facilita la internacionalización mediante el uso de archivos de mensajes, como messages.properties, donde puedes definir textos en diferentes idiomas:

- **Definición de Mensajes**: En messages.properties, define tus mensajes.

  properties

  ```
  saludo=Hola, bienvenido
  despedida=Adiós, gracias por visitar
  ```

- **Uso en Plantillas**: Puedes mostrar estos mensajes con #{}.

  html

  ```
  <p th:text="#{saludo}">Texto por defecto</p>
  ```

### 6. Plantillas Layout Dialect

Thymeleaf también permite crear layouts o diseños base mediante extensiones como **Thymeleaf Layout Dialect**, que funciona de manera similar a las plantillas en otros frameworks (como Blade en Laravel).

- **Incluir Dependencia**:

  xml

  ```
  <dependency>
      <groupId>nz.net.ultraq.thymeleaf</groupId>
      <artifactId>thymeleaf-layout-dialect</artifactId>
      <version>3.0.0</version>
  </dependency>
  ```

- **Estructura de Layouts**: Define un layout base, layout.html, que contenga las secciones comunes:

  html

  ```
  <!DOCTYPE html>
  <html xmlns:layout="http://www.ultraq.net.nz/web/thymeleaf/layout">
  <body>
  ```

```
    <header>Encabezado común</header>
    <div layout:fragment="contenido">Contenido específico aquí</div>
    <footer>Pie de página común</footer>
</body>
</html>
```

- **Extender el Layout**: En cada página específica, extiende el layout e inserta el contenido en las secciones definidas:

  html

  ```
  <html xmlns:layout="http://www.ultraq.net.nz/web/thymeleaf/layout">
  <body layout:decorate="~{layout}">
      <div layout:fragment="contenido">
          <h2>Contenido de la Página</h2>
      </div>
  </body>
  </html>
  ```

## 7. Manejo de Errores de Validación

Para mostrar errores de validación en los formularios, Thymeleaf permite manejar mensajes de error asociados a cada campo.

- **Controlador en Spring**: Puedes enviar errores al modelo cuando falla la validación.

  java

  ```
  if (bindingResult.hasErrors()) {
      return "formulario";
  }
  ```

- **Mostrar Errores en Thymeleaf**: Usa #fields para verificar y mostrar errores específicos en la vista.

  html

  ```
  <form th:action="@{/guardar}" th:object="${persona}"
  method="post">
      <input type="text" th:field="*{nombre}" placeholder="Nombre">
      <div th:if="${#fields.hasErrors('nombre')}"
  th:errors="*{nombre}">Error</div>
      <button type="submit">Guardar</button>
  </form>
  ```

## 8. Manejo de URLs y Recursos Estáticos

- **URLs relativas y absolutas**: Usa @{} para URLs relativas (como recursos CSS o enlaces de navegación).
- **Parámetros en URLs**: Puedes agregar parámetros de forma dinámica.

  html

  ```
  <a th:href="@{/detalle(id=${objeto.id})}">Ver Detalle</a>
  ```

### 9. Bucles y Condicionales Avanzados

- **Bucles**: Puedes iterar sobre listas o mapas y acceder a índices y otras propiedades.

  html

  ```
  <div th:each="item, estado : ${lista}">
      <p th:text="${estado.index}">Índice</p>
      <p th:text="${item.nombre}">Nombre del ítem</p>
  </div>
  ```

- **Condicionales**: Además de th:if y th:unless, se puede usar el operador ternario dentro de expresiones.

  html

  ```
  <p th:text="${usuario.admin ? 'Admin' : 'Usuario'}">Rol</p>
  ```

### 10. Ejemplo Completo Integrando Thymeleaf

Para ver Thymeleaf en acción, aquí tienes una estructura básica:

- **Controlador**:

  java

  ```
  @Controller
  public class HomeController {
      @GetMapping("/home")
      public String home(Model model) {
          model.addAttribute("mensaje", "Bienvenido");
          return "home";
      }
  }
  ```

- **Plantilla (home.html)**:

  html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Thymeleaf Demo</title>
    <link rel="stylesheet" th:href="@{/css/estilos.css}">
</head>
<body>
    <header th:fragment="header">Header aquí</header>
    <main>
        <h1 th:text="${mensaje}">Mensaje</h1>
    </main>
    <footer th:fragment="footer">Footer aquí</footer>
</body>
</html>
```

Estos son conceptos avanzados y útiles de Thymeleaf para crear aplicaciones dinámicas y modularizadas con Spring Boot.

Ejemplos

```
<table>
  <thead>
    <tr>
      <th th:text="#{msgs.headers.name}">Name</th>
      <th th:text="#{msgs.headers.price}">Price</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="prod: ${allProducts}">
      <td th:text="${prod.name}">Oranges</td>
      <td th:text="${#numbers.formatDecimal(prod.price, 1, 2)}">0.99</td>
    </tr>
  </tbody>
</table>
```